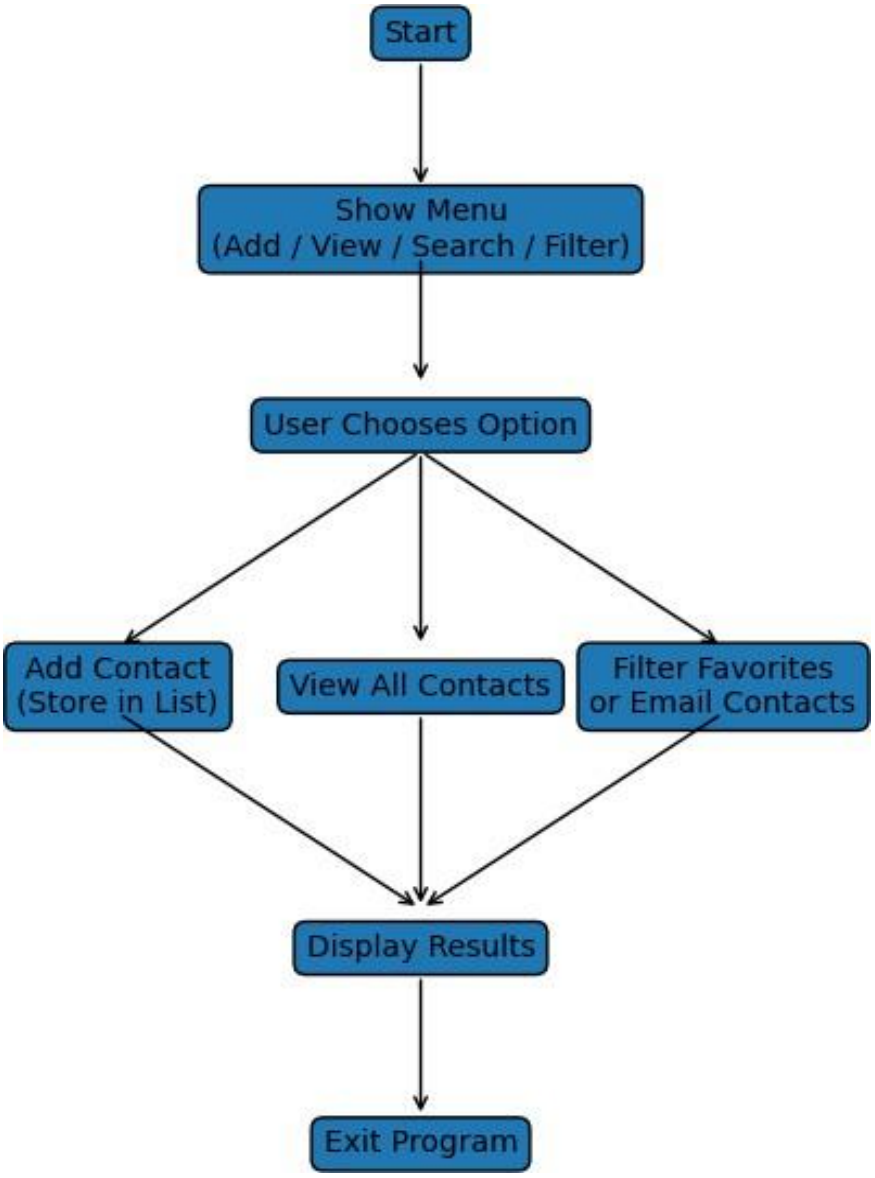
SECTION 4
LAB REPORT

Name = Pawankumar
Course = Android App Development With Gen AI

Console-Based Contact Manager in Kotlin

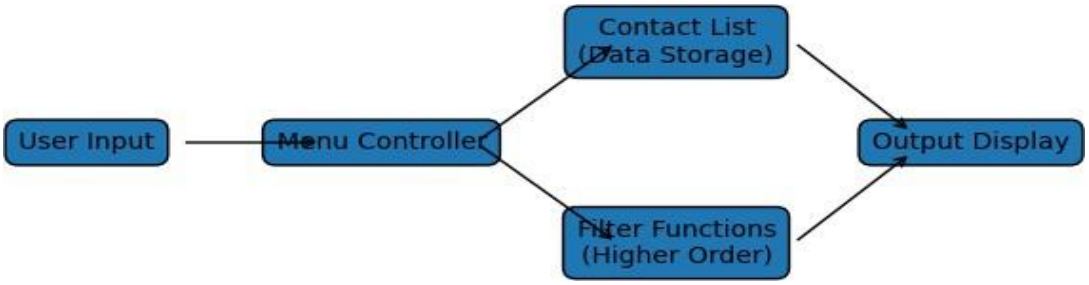
This project demonstrates the development of a console-based Contact Manager using Kotlin. The application allows users to add contacts, view stored contacts, and filter contacts based on specific conditions such as favorite contacts or contacts that contain email addresses. The goal of this project is to practice fundamental Kotlin concepts including: • Data Classes • Null Safety •
Higher-Order Functions • List Filtering • Console-based Program Structure

1. System Flowchart



The flowchart above explains the execution flow of the contact manager program. The program begins with the start node and then repeatedly displays a menu to the user. Depending on the user's selection, the program can add contacts, display contacts, filter favorite contacts, or filter contacts that contain email addresses.

2. System Block Diagram



The block diagram shows the logical structure of the system. The user provides input through the console interface. The menu controller processes the selected operation and interacts with the contact list where all data is stored. Higher-order filtering functions process the list to retrieve specific contacts. The results are finally displayed back to the user.

3. Kotlin Source Code

```
package com.example.businesscard

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
```

```
import androidx.compose.material.icons.filled.Phone
import androidx.compose.material.icons.filled.Place
import androidx.compose.material3.Icon
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.businesscard.ui.theme.BusinessCardTheme
import androidx.compose.ui.tooling.preview.Preview



class MainActivity: ComponentActivity(){
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            BusinessCardTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color=Color (color =0xFFEFBF04)
                ) {
                    BusinessCard()
                }
            }
        }
    }
}
@Composable
fun BusinessCard() {
    Column(
        modifier = Modifier
```

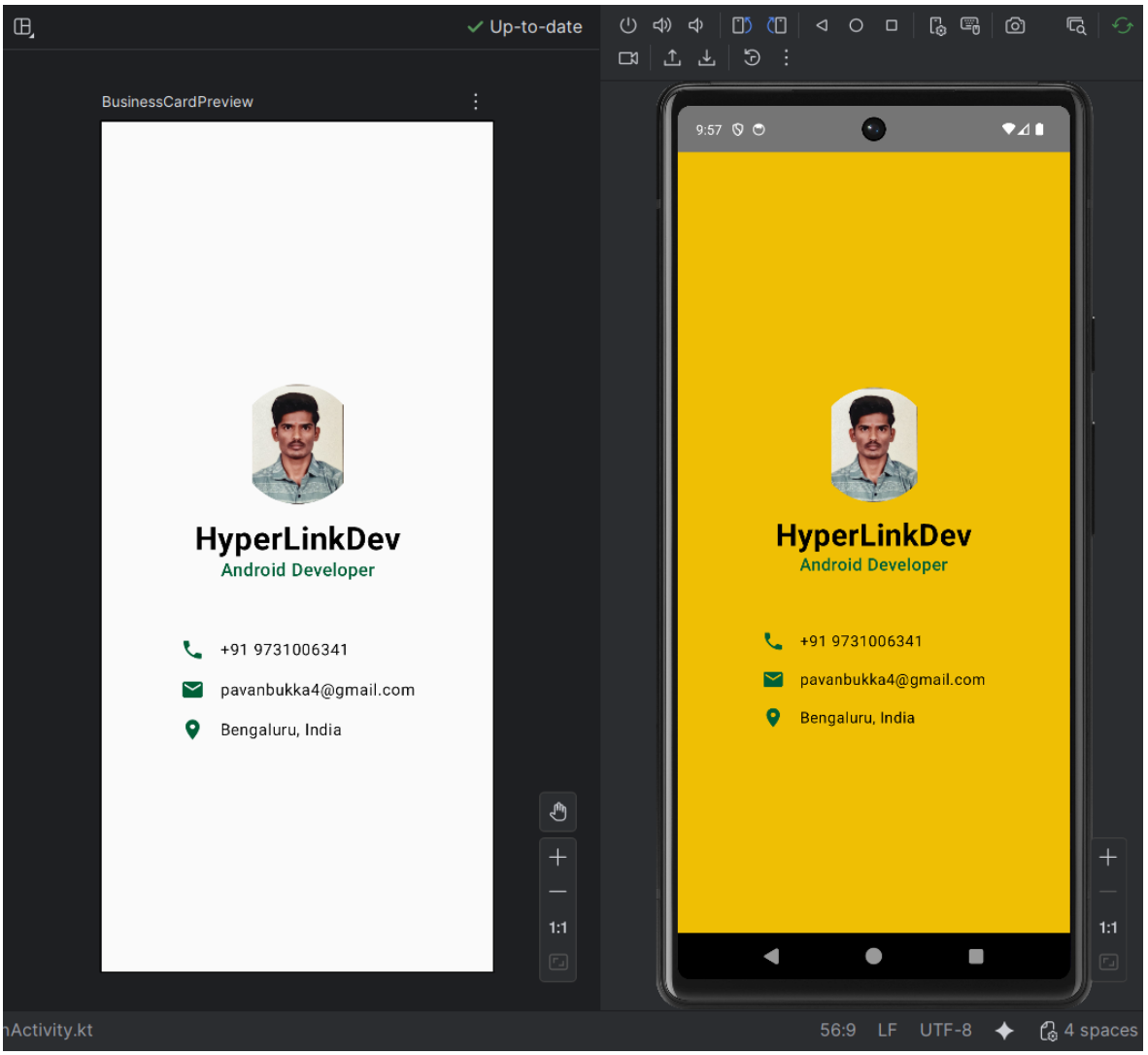
```
            .fillMaxSize()
            .padding(all = 24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Spacer(modifier = Modifier.height(height = 40.dp))
        Image(
            painter = painterResource(id = R.drawable.profile),
            contentDescription = "Profile Image",
            modifier = Modifier
                .size(size = 120.dp)
                .clip(CircleShape)
        )
        Spacer(modifier = Modifier.height(height = 16.dp))
        Text(
            text = "HyperLinkDev",
            fontSize = 32.sp,
            fontWeight = FontWeight.Bold
        )
        Text(
            text = "Android Developer",
            fontSize = 18.sp,
            color = Color(color = 0xFF006038),
            fontWeight = FontWeight.Medium
        )

        Spacer(modifier = Modifier.height(height = 48.dp))
// This Column ensures that the contact rows are aligned to each
other
// while the entire block is centered in the main layout.
        Column(
            horizontalAlignment = Alignment.Start,
            modifier = Modifier.wrapContentWidth()
        ) {
            ContactRow(icon = Icons.Default.Phone, text = "+91
9731006341")
            ContactRow(icon = Icons.Default.Email, text =
"pavanbukka4@gmail.com")
            ContactRow(icon = Icons.Default.Place, text = "Bengaluru,
```

```
India")
        }
    }
}

@Composable
fun ContactRow(icon: ImageVector, text: String) {
    Row(
        verticalAlignment = Alignment.CenterVertically,
        modifier = Modifier.padding(vertical = 8.dp)
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,
            modifier = Modifier.size(size = 24.dp),
            tint = Color(color = 0xFF006038)
        )
        Spacer(modifier = Modifier.width(width = 16.dp))
        Text(
            text = text,
            fontSize = 16.sp,
            color = Color.Black
        )
    }
}
@Preview(showBackground = true)
@Composable
fun BusinessCardPreview() {
    BusinessCardTheme {
        BusinessCard()
    }
}
```

Output :


Up-to-date
BusinessCardPreview
HyperLinkDev
Android Developer
+91 9731006341
pavanbukka4@gmail.com
Bengaluru, India
9:57
HyperLinkDev
Android Developer
+91 9731006341
pavanbukka4@gmail.com
Bengaluru, India
1:1
nActivity.kt
56:9
LF
UTF-8
4 spaces

4. Key Kotlin Concepts Used

- Data Class: Used to store structured contact information such as name, phone, email, and favorite status.
- Null Handling: The email field is nullable (String?) allowing contacts to be created without an email address.

- Higher-Order Functions: The filterContacts function accepts another function as a parameter to dynamically filter data.
- Mutable Lists: Contacts are stored using mutableListOf so that new contacts can be added during runtime.
- Functional Programming: Kotlin's filter() function is used to apply conditions to the contact list.

5. Conclusion

This project demonstrates how Kotlin can be used to build simple but powerful console-based applications. Through the use of data classes, null safety, and higher-order functions, the program maintains clean and reusable logic. The same principles can be extended to larger Android or backend applicatio